

Kruskal's Algorithm

Edge-Based Global Optimization for Spanning Trees

1. Introduction to Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm used to find the **Minimum Spanning Tree (MST)** for a weighted undirected graph. If the graph is disconnected, it finds a **Minimum Spanning Forest** (an MST for each connected component).

Methodology: Unlike Prim's, which grows a single tree from a starting vertex, Kruskal's treats every vertex as an independent tree and progressively merges them by selecting the cheapest available edges across the entire graph.

2. Step-by-Step Logic

The algorithm operates on a global list of edges rather than traversing from node to node:

1. **Sort:** Arrange all edges in non-decreasing order of their weight.
2. **Selection:** Pick the smallest edge. Check if adding it creates a cycle.
3. **Union:** If no cycle is formed, include this edge in the MST and merge the two disjoint sets containing the endpoints of the edge.
4. **Repeat:** Continue until there are $(V-1)$ edges in the MST.

3. The Union-Find Data Structure

To efficiently detect cycles and merge trees, Kruskal's uses a **Disjoint Set Union (DSU)** structure with two key operations:

- **Find:** Determines which set a particular element belongs to. This is used to check if two vertices are already connected.
- **Union:** Merges two disjoint sets into one. Path compression and union-by-rank are used to keep this nearly constant time.

```
def find(self, parent, i):
    if parent[i] == i:
        return i
    return self.find(parent, parent[i])
```

4. Python Implementation Snippet

```
def kruskals_algorithm(self):
    result = []
    # 1. Sort edges by weight
    self.edges = sorted(self.edges, key=lambda item: item[2])
    parent = [node for node in range(self.size)]
    rank = [0] * self.size

    for u, v, weight in self.edges:
        x = self.find(parent, u)
        y = self.find(parent, v)
        # 2. If roots are different, no cycle
        if x != y:
            result.append((u, v, weight))
            self.union(parent, rank, x, y)
    return result
```

5. Time Complexity and Usage

Kruskal's performance is dominated by the sorting step:

Complexity: $O(E \log E)$ or $O(E \log V)$. Since E can be at most V^2 , $\log E$ is proportional to $\log V$.

When to use: Kruskal's is generally faster for **sparse graphs** (graphs with fewer edges) because it focuses on edges rather than vertices. It is also the go-to choice when you need a Spanning Forest for disconnected graphs.

CodeHive

Forests and Disconnected Components (Part 1)

A unique advantage of Kruskal's algorithm is its ability to naturally handle disconnected graphs. While Prim's algorithm requires a manual "restart" to find fragments of a disconnected graph, Kruskal's logic ensures that it finds a **Minimum Spanning Forest** without any extra code modifications.

Stopping Criterion: In a connected graph, we can stop Kruskal's once $(V-1)$ edges are added. For a graph with k connected components, the search concludes once $(V-k)$ edges are added to the forest.

Rank and Path Compression

The efficiency of the DSU structure is improved by **Path Compression** (making nodes point directly to the root during a find operation) and **Union by Rank/Size** (attaching the shorter tree to the taller one). These optimizations make the DSU operations run in nearly inverse Ackermann time, essentially $O(1)$ in practice.

Forests and Disconnected Components (Part 2)

A unique advantage of Kruskal's algorithm is its ability to naturally handle disconnected graphs. While Prim's algorithm requires a manual "restart" to find fragments of a disconnected graph, Kruskal's logic ensures that it finds a **Minimum Spanning Forest** without any extra code modifications.

Stopping Criterion: In a connected graph, we can stop Kruskal's once $(V-1)$ edges are added. For a graph with k connected components, the search concludes once $(V-k)$ edges are added to the forest.

Rank and Path Compression

The efficiency of the DSU structure is improved by **Path Compression** (making nodes point directly to the root during a find operation) and **Union by Rank/Size** (attaching the shorter tree to the taller one). These optimizations make the DSU operations run in nearly inverse Ackermann time, essentially $O(1)$ in practice.